

YAML files. We must begin treating our machine learning data expectations with the exact same level of rigour. Data contracts must be codified, version-controlled, and injected as context alongside the application source code.

Open source tools have paved the way for this. When a model is trained or versioned, the metadata regarding its input schema and preprocessing requirements is often stored as JSON or YAML artifacts. A context-aware system acts as the enforcement mechanism for these contracts. It bridges the gap by reading these declarative artifacts and cross-referencing them against the application code during the review process.

For instance, if a version-controlled data contract specifies that a missing account balance must trigger a payload rejection, the context-aware reviewer will scan the backend API implementation. If it finds that the API is carelessly defaulting the balance to zero to prevent an application crash, it will block the deployment. This ensures that the infrastructure executing the code is always perfectly synchronised with the mathematical assumptions of the model.

A real-world context failure: API contract drift in LLMs

To truly understand the value of context-aware systems, consider a real-world open source deployment scenario involving generative artificial intelligence. A team is building a natural language processing service using an open source large language model, serving it on Linux servers equipped with GPUs.

Language models are highly sensitive to text encoding and tokenization. The model expects input text to be strictly sanitised: HTML tags removed, Unicode characters

normalised to UTF-8, and trailing whitespace stripped. The data science team documents these exact contextual requirements in a GitLab epic and implements a Python script utilising regular expressions to clean the training data.

Months later, a frontend developer updates the client web application, inadvertently changing the way text is encoded before it is sent to the API, falling back to ISO-8859-1. The backend engineer writes an API wrapper that accepts the payload without throwing an HTTP bad request error and passes the raw, improperly encoded bytes directly to the language model.

Because the API contract drifted and the context was lost, the model began receiving text filled with unescaped HTML entities and broken Unicode characters. The GPU does not crash; it simply processes the bad tokens and generates degraded, nonsensical text. In a traditional workflow, finding this bug requires days of digging through application logs, using network analysis tools to examine raw payloads, and tracing the execution path from the load balancer down to the Python application.

In a context-aware workflow, the issue is flagged the moment the backend engineer attempts to merge the API endpoint without including the UTF-8 sanitisation step defined in the original architecture. The context-aware reviewer cross-references the pull request against the original project guidelines, reads the original data science requirements from the ticket, and points out the missing logic. It turns a multi-day production nightmare into a five-minute code review fix, proving the immense value of maintaining contextual state throughout the deployment lifecycle.

Observability as the second line of defence

Even with the best context-aware review systems in place at the continuous integration level, production environments are chaotic. Upstream data sources change, client applications send unexpected payloads, and data drift occurs naturally over time. Therefore, context-aware deployment must be paired with robust, open source observability to monitor the runtime context.

When deploying machine learning models, tools like Prometheus and Grafana are essential for catching the silent failures that manage to slip through. By exposing custom metrics from your machine learning API, you can monitor not just system health—like CPU usage or HTTP latency—but data health. You can track the rate of missing values, the distribution of input features, and the variance in model predictions to ensure the runtime context matches the assumed context.

If the mean value of an input feature suddenly shifts by three standard deviations, Prometheus can trigger an alert. This allows the engineering teams to investigate whether the shift is due to a genuine change in user behaviour, or a silent pipeline failure caused by an upstream API contract drift. Context-aware review systems prevent these errors from entering the codebase at the pull request stage, while open source observability tools ensure that external factors do not quietly violate the context at runtime.

Redefining ‘done’

Ultimately, the goal of code review, automated continuous integration pipelines, and DevOps culture



Figure 3: Monitoring real-time data drift and missing features in production pipelines